



Construcción de software y toma de decisiones (Gpo 501)

**Competencia STC205: Elaboración de casos
de prueba de software Intento 2**

Carlos Arturo Gómez Ayala - A01711027
ITESM Campus Querétaro
Profesor: Enrique Alfonso Calderón Balderas
Co-Titular: Alejandro Fernández Vilchis
Co-Titular: Denisse Lizbeth Maldonado Flores

Fecha: 8/06/2026

Elaboración de Pruebas de Software

* Las Pruebas de Software son 30-50% del costo\$

¿Qué es una Prueba?

Es una verificación automática de que tu código produce el resultado esperando para una entrada.

Si la salida real coincide lo esperado ✓✓

Si no pasa la prueba → Falla y revela defecto

Rational Unified Process (RUP): Mejor comenzar a probar lo más pronto ASAP posible.

[Use Cases → Test cases]

Como se sabe los casos de uso determinan y definen los requerimientos funcionales del Software.

RUP: "Los casos de uso describen completamente la secuencia de acciones hechas por el sistema para dar un resultado observable resultando en valor para una persona o a otro sistema usando el producto".

1er Fundamental

2do

Test cases → Test Precedimientos → Test Scripts ^{td}

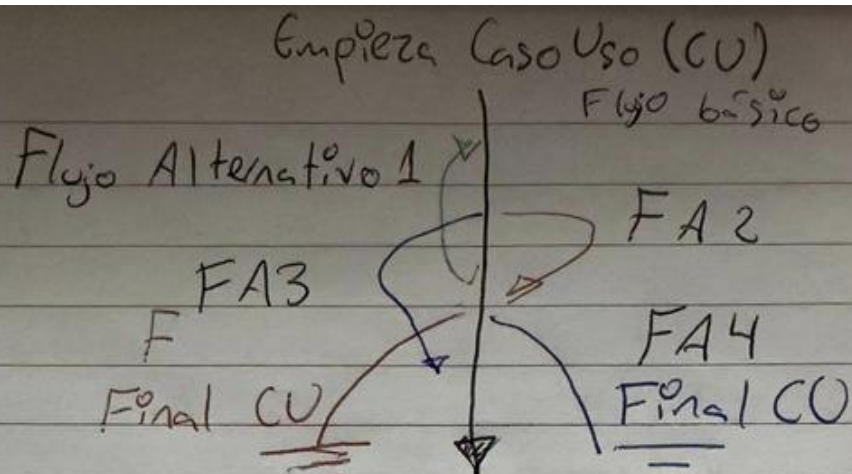
¿Por qué?

Test Cases: Identifican y comunican las condiciones que se van a implementar y son necesarias para verificar exitosas y aceptables implementaciones de los requerimientos productos.

Tabla Formato Use Cases

Nombre	Apropiado
Breve Descripción	Rol y Propósito
Flujo de Eventos	Lo que hace en ese caso de uso, no como resuelve específicamente. La debe entender el cliente
Requerimientos Special	Colecta todos los RF, RNF, que no se consideran en el caso de uso, pero de los que se debe considerar para diseño o implementación
Precondiciones	Lo que se necesite antes que empiece el caso de uso
Post Condición	

Lo más Importante: Flujo de Eventos.

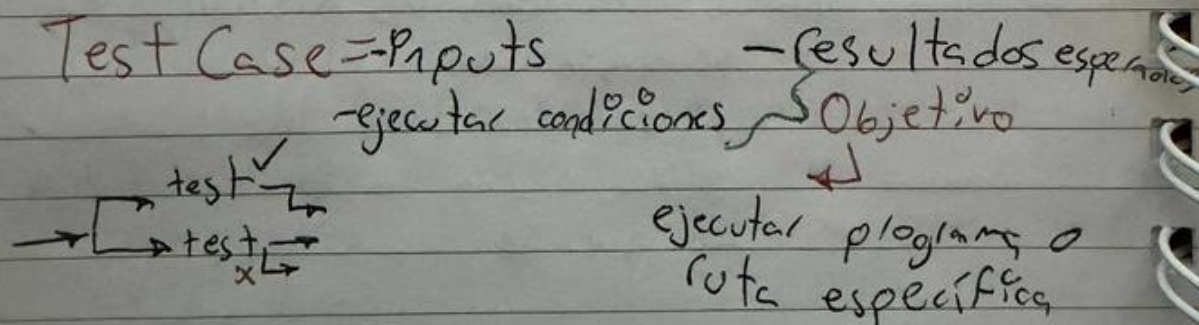


Ejemplos de Enroll en Iris Tec Monterrey.

Tabla Escenarios Casos de Uso (CU)

Escenario 1	Basic Flow
Escenario 2	Basic Flow Alternativa 1
Escenario 3	Basic Flow Alternativa 2 y 1
Escenario 8	Basic Flow $\forall$ Alternativa 1 a la 8, etc.

Generar Test Cases



RECORDAR: El propósito es de identificar y comunicar condiciones que se van a implementar. Test cases son necesarios para exitosas y aceptables implementaciones de los requerimientos.

Términos a usar

- Error: Acción humana incorrecta (Programador, tester, etc).
- Defecto: Bug en el código que resulta del error.
- Falla: Lo que el usuario observa cuando ejecuta.

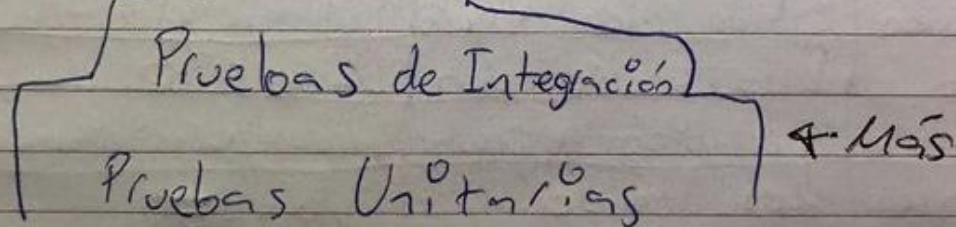
Caso de Prueba: Una entrada específica + el resultado que se espera al ejecutar.

Test suite: Conjunto casos relacionados (Ej. todas las pruebas de personaje).

Oráculo: Criterio que decide si el resultado es correcto o no.

Automatización: Que las pruebas se ejecuten solas, no con humanos.

ETG/UI



Unitarias: Una función o clase aislada.

Integración: Varios módulos trabajando juntos.

Sistema E2E: Sistema completo extremo a extremo.

Aceptación: Verifica que cumple los requisitos del cliente.

Usabilidad: Personas reales usando el producto.

Regresión: Que no rompió lo que ya funcionaba.

Verificación vs Validación

Verificación	Validación
¿Construimos el producto correctamente? el código funciona?	¿Construimos lo que quiere el cliente? el producto correcto?
- Unitarias - Integración - P. sistemas	- P. usabilidad. - P. en voz alta. - P. aceptación

Anatomía Caso Prueba

Campo	¿Qué es?	Ejemplo
ID	Identificador	TC-001
Descripción	¿Qué prueba?	Recibir daño = vida ↓
Precondición	Estado inicial necesario	Personaje vida = 100
Entrada	Datos que se le dan	Daño = 30
Acción	Operación a ejecutar	personaje recibir daño (30)
Salida esperada	Resultado correcto	vida = 70

Clases de Equivalencia

Agrupar entradas que se traten igual. Si una funciona las demás también

X	✓	X
< 18 años	$18 \leq \text{edad} \leq 130$	$\text{edad} > 130$
no vota	válido a votar, etc entre esos datos	

3 casos de Prueba cubren toda la lógica

Introducción

Antes de este curso hacía lo que creo que hace todo el mundo cuando esta programando y era que imprimía valores, mandaba peticiones y abría el navegador para ver si algo explotaba o no funcionaba o que se imprimía. Eso funciona para detectar errores obvios y finalmente era prueba y error, pero tiene un problema fundamental y es que no es repetible y algo que se me ha quedado mucho de la clase, es ser metódico para no tener que estar arreglando errores 'quirúrgicamente' después, costando recursos y costando tiempo. Si modifico una función hoy, mañana tendría que volver a hacer todo eso manualmente para saber si sigo sin romper nada y al día siguiente otra vez y así. Además, ese conocimiento de qué probé vive solo en mi cabeza, mi equipo no lo ve y no hay registro de ello, entonces nadie sabría que falla o fallaría sin pruebas mas que la persona que lo hizo.

Error, defecto y falla

Estas tres palabras parecen sinónimos pero no lo son. Un error es la acción humana incorrecta, lo que hizo el programador mal, por ejemplo escribir `vidaActual +` dañado en lugar de restarle. Ese error produce un defecto, que es el bug concreto que queda en el código esperando ser ejecutado. Y cuando alguien finalmente ejecuta ese código y ve algo raro en pantalla eso es la falla, lo que el usuario observa.

Verificación vs. validación

Esta distinción me parece de las más importantes del curso. Verificación pregunta si construimos el producto correctamente, mira hacia adentro ¿el código hace lo que dice que hace? Ahí caen las pruebas unitarias, de integración y de sistema. Validación pregunta si construimos el producto correcto, mira hacia afuera: ¿es lo que el usuario realmente necesita? Ahí caen las pruebas de usabilidad, las de pensamiento en voz alta y las de aceptación, ambas son importantes para llevar una programación exitosa y un producto de valor al cliente.

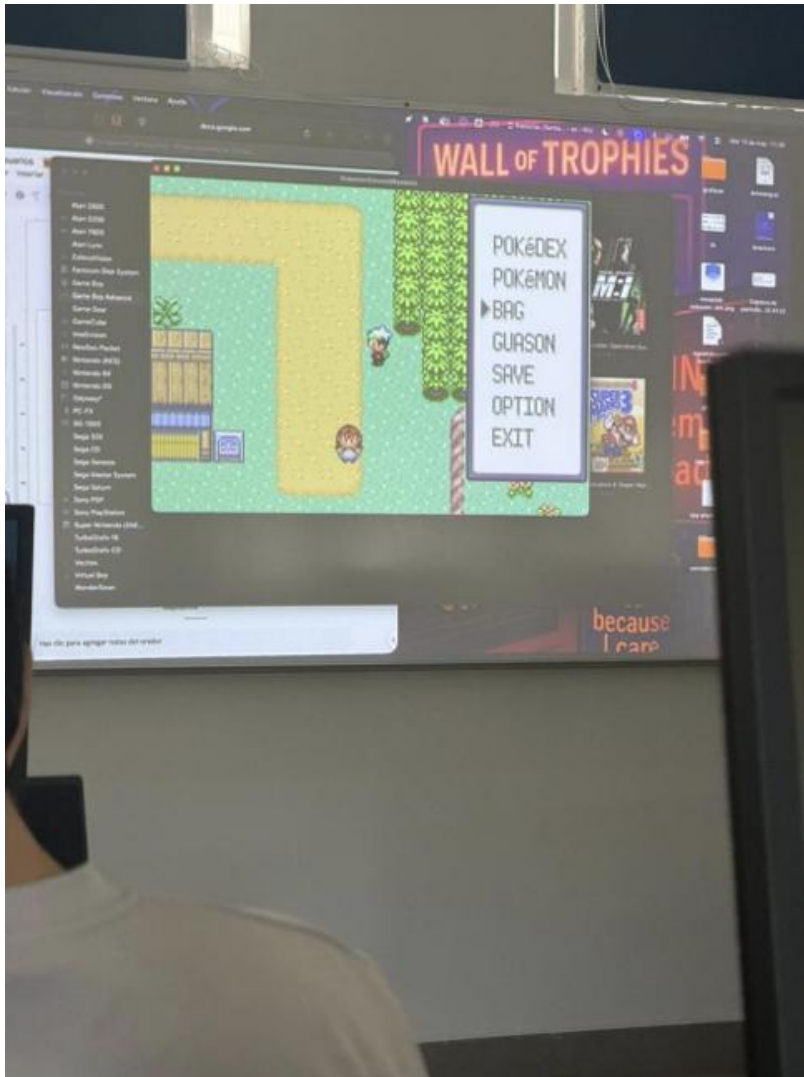
La pirámide de pruebas

en la base van muchas pruebas unitarias porque son rápidas, baratas y aíslan defectos con precisión. En el medio van las de integración. Y en la punta, pocas pruebas E2E porque son lentas, frágiles y cuando fallan no siempre te dicen dónde está el problema exactamente.

Anatomía de un caso de prueba

Antes de tocar cualquier framework hay que saber qué probar. Un caso bien diseñado tiene seis campos: un ID para identificarlo, una descripción de qué se está probando, una precondition que define el estado inicial necesario, la entrada con los datos que se le pasan, la acción que se va a ejecutar y la salida esperada. Sin esos seis elementos, un caso de prueba es ambiguo y difícil de repetir.

Pruebas de pensamiento en voz alta (Think Aloud)



Esta no la olvido por nada del mundo y que jugadores pusieron de Mario y Pokemon Emerald con un mod randomizer de Pokemon. En fin, esta es una técnica de validación, no de verificación. Mientras el usuario usa el sistema, va hablando en voz alta diciéndote qué piensa, qué espera que va a pasar y qué le confunde o como va progresando, etc. No intervienes, solo observas y tomas notas como el ejemplo de clase. El punto no es probar que el código funciona, que eso ya lo cubren las unitarias, sino descubrir si la interfaz y el flujo tienen sentido

para alguien que no diseñó el sistema. **Durante la entrevista de la competencia, se realizó de forma práctica una prueba de pensamiento en voz alta junto con el equipo**, donde se verbalizó el razonamiento al momento de analizar el caso planteado por el profesor, lo que se esperaba que pasara, qué entradas se usarían y el por qué y qué salida confirmaría que el sistema funcionaba correctamente.

Lo que más te revela es exactamente lo que el usuario dice antes de cometer un error, porque ahí está la brecha entre lo que tú diseñaste y lo que él entiende. Si alguien dice 'supongo que este botón hace X' y hace clic esperando X y le pasa Y, eso es un hallazgo de usabilidad que ninguna prueba automatizada te daría, como el botón de salto cambiando de letra, pero siendo el botón de abajo y que se cambie ahora al botón de arriba de 4 botones. El punto es ver y documentar esas interacciones.

Pruebas de integración

Una prueba unitaria verifica una función aislada del resto. Una prueba de integración verifica que varios módulos funcionan correctamente cuando trabajan juntos. La diferencia importa porque dos módulos pueden pasar sus pruebas unitarias por separado y aun así romperse cuando se llaman entre sí ya sea por incompatibilidades en los datos que se pasan, por estados compartidos que no se limpian entre pruebas o por dependencias externas como las bases de datos.

Valores frontera

Los bugs viven en los bordes. El error más clásico del mundo es escribir $<$ cuando debería ser $\leq$ o viceversa. Por eso la técnica de valores frontera dice que hay que probar justo abajo del límite, exactamente en el límite y justo arriba del límite. Si la frontera es 18, pruebo 17, 18 y 19, no 5 y 50.

Aplicación al proyecto CoreData

El proyecto en el que estamos trabajando es un sistema web de cumplimiento PLD desarrollado para SVA LAW, una firma legal fintech que opera como SOFOM bajo regulación CNBV. El sistema maneja el flujo operación → alerta → reporte y tiene más de veinte requerimientos funcionales implementados en Node.js con Express, PostgreSQL y despliegue en Vercel. En términos de las pruebas, la actividad de integración que documentamos fue una revisión funcional de las vistas principales del sistema. Evaluamos el menú dashboard, el módulo de operaciones, el de alertas y el listado de clientes. El criterio fue si cada sección

mostraba los datos correctos, si los controles respondían como se esperaba y si había consistencia visual en colores y tipografía y demás. Los resultados mostraron que funcionalidades como el reordenamiento de columnas y la paginación de clientes operaban correctamente, mientras que otras como los filtros de operaciones y la lista de personas bloqueadas aún estaban pendientes por implementarse o que no pudiéramos colocar esos filtros donde quisiéramos. Esta revisión es un ejemplo concreto de pruebas manuales de integración a nivel de sistema, donde verificas que los módulos del frontend y del backend funcionan en conjunto para el usuario final. Complementa las pruebas unitarias que se hicieron con Jest sobre los módulos de lógica de negocio. Los conceptos de clases de equivalencia y valores frontera son aplicables al sistema por ejemplo, en la validación del RFC de un cliente, la clase válida tiene un formato específico de 13 caracteres alfanuméricos según el SAT, la clase inválida incluye formatos distintos y los valores frontera son cadenas de 12 y 14 caracteres para detectar errores de conteo. El mismo razonamiento aplica a campos como CURP, montos de operación y fechas, todos con reglas precisas de la regulación CNBV que hacen que los casos de prueba puedan derivarse directamente de los requisitos.

Por lo que, se deben de hacer los casos de prueba o de test lo más pronto posible como menciona el artículo, ir de la mano de requerimientos al mismo tiempo las pruebas, ahorraría bastante tiempo y trabajo para después. Se cometen menos defectos o fallas si no se cometen errores y da como resultado un mejor producto porque se tuvo una metodología previa y se ejecuta de mejor manera. Referencia: Generating Test Cases From Use Cases by Jim Heumann. Requirements

Management Evangelist Rational Software Material de clase.

Scenario Name	Starting Flow	Alternate

Scenario 1 - Successful registration	Basic Flow	
Scenario 2 - Unidentified student	Basic Flow	A1
Scenario 3 - User quits	Basic Flow	A2
Scenario 4 - Course catalog system unavailable	Basic Flow	A4
Scenario 5 - Registration closed	Basic Flow	A5
Scenario 6 - Cannot enroll	Basic Flow	A3

Mi participación e involucramiento en las pruebas del proyecto

En mi participación en el proyecto, se adjunta mi participación en algunos casos de prueba de entregas anteriores correspondiendo a las entregas a partir del "Avance 7": Me concentrare para esta competencia en las secciones 2.8 a 2.12 del Avance 9 del proyecto CoreData, que corresponden al 20% a 30% (aproximadamente) adicional de casos de prueba solicitados, en estas fui el responsable directo de esos procesos identificar qué módulos no tenían cobertura, diseñar los casos de prueba, en definir los códigos de Jest y ejecutarse hasta que todos pasaran sin errores.

Qué identifiqué y por qué los elegí

Revisé las pruebas existentes de los avances 6,7 y avance 8 y detecté que varios módulos importantes del sistema no tenían aún ninguna prueba automatizada como ValidarDocumento, ImportarCSV, EditarCliente, ApiListaClientes, y el modelo completo de alertas con sus funciones ObtenerAlertas, ResolverAlerta, CambiarEstatus, ObtenerAlertaPorId y GenerarAlertaSiAplica. Elegí esos porque son módulos del core de las necesidades del negocio de SVA LAW en materia de PLD, su falta de cobertura representaba un riesgo real para el proyecto.

Se adjunta video de las pruebas realizadas y documentadas

<https://drive.google.com/file/d/1LglnB3qN5gYgsqU7aFNB87wevXG4hKYE/view?usp=sharing>

Cómo se diseñaron los casos de prueba

Para diseñar los casos de prueba del 20% y extra adicional, me apoyé en la inteligencia artificial como herramienta de ideación y apoyo, usando el prompt documentado en las referencias del Avance 9. Le proporcioné el contexto del proyecto, la arquitectura del modelo-visual-controlador MVC, los módulos sin cobertura y los requisitos específicos de cada función y es a partir de eso que generé una propuesta inicial de casos. Sin embargo, lo importante es que no los acepté tal cual, como se los ha instruido en el manejo de la IA, los revisé contra el código real del sistema para verificar que los flujos alternativos (o escenarios) y las salidas esperadas fueran las correctas y se les hizo el ajusté donde era necesario. Por ejemplo, para ValidarDocumento la herramienta propuso casos para falta de ID, estado inválido y rechazo sin motivo, que corresponden a los flujos alternativos del caso de uso. Los validé contra el controller real y confirmé que las respuestas y los mensajes de error coincidieran con lo implementado. Para GenerarAlertaSiAplica apliqué la técnica de valores frontera vista en clase sobre los umbrales del sistema PLD, que estas reglas de momento son ejemplos de reglas configurables que la empresa puede modificar desde el módulo de Reglas. Probé un monto por debajo del primer umbral de ejemplo, uno entre los dos umbrales de ejemplo y uno por encima del mayor, que es exactamente el mismo razonamiento del ejemplo que se nos dio de clase en la dispositiva de “Valores Frontera”, con el rango de edad de 18 a 130, *se prueban los bordes donde viven los bugs y se intenta quebrar/romper el sistema*, es una manera de resumir la metodología de casos de prueba, que se nos dio al recibir retroalimentación de la primera entrevista.

2.8 Validación de documentos del cliente

ID	Descripción	Precondición	Entrada	Acción	Salida esperada
CP-1	Sin idDocumento da el → error 400	Servidor corriendo	idDocumento = vacío, estado = Validado	POST /clientes/validar-doc umento	HTTP 400 con { exito: false } El servidor rechaza la petición por falta de ID
CP-2	Sin estado es→ error 400	Servidor corriendo	idDocumento = 1, estado = vacío	POST /clientes/validar-doc umento	HTTP 400 con { exito: false }El servidor rechaza la petición por falta de estado
CP-3	Estado inválido da → error 400	Servidor corriendo	idDocumento = 1, estado = Aprobado	POST /clientes/validar-doc umento	HTTP 400, mensaje "Estado inválido" El servidor rechaza porque "Aprobado" no es un estado válido
CP-4	Rechazar sin motivo aparente = → error 400	Servidor corriendo	idDocumento = 1, estado = Rechazado, motivo = vacío	POST /clientes/validar-doc umento	HTTP 400, mensaje "Debe indicar el motivo" El servidor rechaza porque no se puede rechazar sin explicar por qué
CP-5	Se va a validar documento exitosamente	Supabase disponible	idDocumento = 5, estado = Validado	POST /clientes/validar-doc umento	HTTP 200 con { exito: true } El documento queda marcado

Fragmento de sección de pruebas de "Avance 9: Versión Beta- CoreData Consulting"

2.12 Generación automática de alertas PLD

ID	Descripción	Precondición	Entrada	Acción	Salida esperada
CP-1	Monto >= 1,500,000 sin reglas activas → alerta nivel Alto (estas se pueden modificar)	Sin reglas activas en BD	idoperacion=10, monto=2000000	Llamar GenerarAlertaSi Aplica()	INSERT en tabla alerta con nivel = "Alto" Se crea una alerta de nivel Alto por superar en este ejemplo de regla \$1,500,000
CP-2	Ejemplo usado de Monto entre 600,000 y 1,499,999 → alerta nivel Medio	Sin reglas activas en BD	idoperacion=12, monto=800000	Llamar GenerarAlertaSi Aplica()	INSERT en tabla alerta con nivel = "Medio" Se crea una alerta de nivel Medio por estar dentro del ejemplo de regla entre \$600,000 y \$1,499,999
CP-3	Ejemplo usado de Monto < 600,000 sin reglas activas → no se genera alerta	Sin reglas activas en BD	idoperacion=11, monto=50000	Llamar GenerarAlertaSi Aplica()	No se ejecuta INSERT en tabla alerta. No se genera ninguna alerta porque el monto es bajo
CP-4	ApiResolverAlerta sin idAlerta → error 400	Servidor corriendo	body = { resolucion: "Sin ID" }	POST /api/alertas/resolver	HTTP 400 con { msg: "Falta idAlerta" }. El servidor rechaza porque no sabe qué alerta resolver
CP-5	ApiCambiarEstatus exitoso	Supabase disponible	idAlerta = 3, estatus = "En revision"	POST /api/alertas/estatus	JSON con { exito: true }.El estatus de la alerta cambia correctamente

Fragmento de sección de pruebas de "Avance 9: Versión Beta- CoreData Consulting"

Cómo implementé las pruebas en Jest

Se escribieron dos archivos en la carpeta tests/ del proyecto, clientes.test.js con 11 casos y alertas.test.js con 12 casos. En ambos se utilizó jest.mock para simular Supabase y que las pruebas no intentaran conectarse a la base de datos real del sistema. También tuve que 'mockear' el modelo de historial porque ValidarDocumento llama a RegistrarAccion después de cada operación y si no lo mockeaba aparecían errores en consola, aunque los tests pasaran correctamente lo cual no es del todo correcto. Usé el patrón AAA con comentarios en cada sección para que cualquier persona del equipo que abra el archivo entienda qué está probando cada caso sin tener que adivinar. También, cabe resaltar que he aprendido muchas abreviaturas importantes en el mundo de la Ingeniería de Tecnologías Computacionales, como RBAC, ACID y para este caso la de AAA.

Arrange-¿Qué necesito preparar?

Act-¿Qué voy a probar?

Assert-¿El resultado fue correcto?

Qué se corrigió durante la ejecución de las pruebas

La primera vez que corrí en la terminal `npx jest --coverage` aparecieron mensajes de error en consola del modelo de historial. Identifiqué que el problema era que RegistrarAccion pues este intentaba usar Supabase, que estaba mockeado solo en la configuración pero no en el modelo del sistema. Lo resolví agregando el mock del modelo de historial al inicio del archivo de clientes.test.js.

```
1. jest.mock('./models/historial.model', () => ({  
    RegistrarAccion: jest.fn() }));  
2. jest.mock('./config/supabase', () => ({  
    from: jest.fn() }));
```

Después de eso los 23 casos pasaron sin errores ni warnings, como se ve en los videos de evidencia del avance. Ese proceso de encontrar el error, entender de dónde venía y resolverlo fue en sí mismo un ejercicio de las habilidades de la competencia.

Fragmentos de los códigos de pruebas

alertas.test.js

...

// ——— CP-1: ObtenerAlertas retorna lista

```
test('CP-1 ObtenerAlertas: retorna exito true con array de alertas', async () => {
  const fakeAlertas = [
    { idalerta: 1, nivel: 'Alto', estatus: 'Pendiente', idcliente: 3 }
  ];
  supabase.from.mockReturnValue({
    select: jest.fn().mockReturnValue({
      order: jest.fn().mockResolvedValue({ data: fakeAlertas, error: null })
    })
  });

  const resultado = await modelAlertas.ObtenerAlertas();

  expect(resultado.exito).toBe(true);
  expect(resultado.alertas).toHaveLength(1);
});
...
```

clientes.test.js

...
// ——— CP-1: ValidarDocumento sin idDocumento

```
test('CP-1 ValidarDocumento: sin idDocumento retorna 400 con exito false', async () => {
  const req = { body: { estado: 'Validado' }, ip: '127.0.0.1' };
  const res = crearResMock();

  await controller.ValidarDocumento(req, res);

  expect(res.status).toHaveBeenCalledWith(400);
  expect(res.json).toHaveBeenCalledWith(expect.objectContaining({ exito: false }));
});
```

// ——— CP-2: ValidarDocumento sin estado

```
test('CP-2 ValidarDocumento: sin estado retorna 400 con exito false', async () => {
  const req = { body: { idDocumento: '1' }, ip: '127.0.0.1' };
  const res = crearResMock();

  await controller.ValidarDocumento(req, res);
```

```
expect(res.status).toHaveBeenCalledWith(400);  
expect(res.json).toHaveBeenCalledWith(expect.objectContaining({ exito: false  
}));  
...
```

Al recordar lo visto y aprendido en clases, los tests utilizan la estructura de test() y de expect().

NOTA: Los archivos alertas.test.js y clientes.test.js están diseñados específicamente para correr dentro del repositorio de CoreData Consulting, ya que los requires apuntan directamente a los controllers y modelos del proyecto. Para correrlos correctamente hay que colocarlos en la carpeta tests/ del proyecto y ejecutar npx jest --coverage desde la raíz de este, tal como se muestra en los videos de evidencia del avance.

Resultados obtenidos y reflexión de competencia ‘Elaboración de Pruebas de Software’

Al correr en la terminal de Visual Studio el comando de “npx jest --coverage” con los archivos nuevos con las pruebas que fueron implementadas para este ‘Avance 9’, se lograron implementar las nuevas pruebas y lograr el porcentaje de avance para esta sección del proyecto, hay partes del sistema que ahora tienen una verificación repetible, documentada y que cualquier miembro del equipo o de personas externas que puede correr en segundos.

Verlo así en la terminal fue satisfactorio y no porque los números sean perfectos, sé que hay líneas sin cubrir y módulos que no se alcanzaron, sino porque en el proyecto había algo concreto y medible que nos daba un indicio de ‘esto funciona como debía’ y que se puede demostrar sin tener que abrir el navegador y rezar. Eso se siente diferente a cuando simplemente pruebas a mano y confías en que nada se rompió, nuevamente la metodología ganando.

Lo que más me quedó de ver esos resultados es entender por qué vale la pena el esfuerzo de hacer esto de las pruebas bien desde el principio, ya que al momento de estudiar para la entrevista con distintas fuentes de información como el de “Requirements Management Evangelist Rational Software” y también por enseñanza de los profesores, es que se recomienda ir de la mano de implementación del código y de elaboración de prueba, es decir, comenzar las pruebas tan pronto como sea posible.

En el proyecto hubo momentos en que hacíamos cambios en el código y no sabíamos exactamente qué podíamos haber roto, simplemente revisábamos las pantallas principales y si no había nada obvio, seguíamos. Con las pruebas eso cambia, porque si algo se rompe silenciosamente Jest te lo dice antes de que llegues a la presentación con el cliente o antes de que el socio formador lo vea. En la recta final del proyecto, cuando los cambios son más frecuentes y delicados o “quirúrgicos” eso vale mucho. También me di cuenta de que el porcentaje de cobertura no es la meta en sí misma. La meta es tener confianza en lo que se entrega y en recordar la pirámide de pruebas, pues por costos no podrá haber 100% para todas las funcionalidades, pero si para lo que mas importa para generar valor a los clientes.

Antes de esta Unidad de formación, mi forma de pensar radicaba en el ‘probar y error’ era abrir el navegador, hacer clic en las cosas y ver si algo tronaba/rompía. Si no tronaba asumía que estaba bien. Eso lo hacía de forma natural sin cuestionarlo, creo que es lo que hace la mayoría cuando empieza a programar y no se enseña de otra forma al inicio. El problema es que eso no es probar, es confiar en que no hay errores.

Lo que más me cambió de esta competencia no fue aprender a usar Jest, que eso al final es una herramienta, sino entender el por qué las pruebas existen. La distinción entre verificación y validación me pareció de las más útiles de todo el semestre. Verificación es preguntarte si construiste el producto bien, si el código hace lo que dice que hace. Validación es preguntarte si construiste el producto correcto, si lo que entregaste resuelve lo que el cliente realmente necesita. Yo antes solo hacía verificación sin saberlo y ni siquiera la hacía bien porque era manual y no quedaba registro de nada.

En el proyecto CoreData esa distinción se volvió muy concreta. Podíamos tener todas las pruebas pasando y aun así entregar algo que SVA LAW no entiende o que no encaja con cómo ellos trabajan en su día a día como SOFOM. Cada reunión con el cliente fue en realidad una prueba de validación y cada cosa que señalaron que no les quedaba clara fue un hallazgo que ningún Jest hubiera detectado. Eso me hizo entender que las pruebas automatizadas y el contacto con el cliente no son sustitutos uno del otro, sino que son complementarios.

Lo que me llevo para lo que sigue es que probar bien desde el principio ahorra mucho más tiempo del que cuesta. En el proyecto lo viví al revés, implementamos las pruebas tarde. Si las pruebas hubieran estado desde antes, esa incertidumbre no hubiera existido, pero son conocimientos que se debían tocar durante el semestre por lo que con lo que me voy, es algo que voy a aplicar diferente en el

siguiente proyecto, no como un trámite de entrega sino como parte del proceso desde el día uno, que es ir generando requerimientos y diseños de software, junto con las pruebas tan pronto como sea posible al terminar alguna implementación. La eficiencia, generar valor, atender a necesidades; se dan las herramientas para solventar y tomar la mejor decisión para cada proyecto o cliente que como futuros ingenieros podamos tener.

Referencias

Generating Test Cases From Use Cases by Jim Heumann. Requirements Management Evangelist Rational Software

Material de clase.

De la Cruz Ambrosio, Luis Enrique; Martinez Barragan, Luis Fernando; y Gómez Ayala, Carlos Arturo. (05 de junio de 2026). *Avance 9: Versión Beta*. CoreData Consulting — Construcción de Software y Toma de Decisiones, Grupo 501. Instituto Tecnológico y de Estudios Superiores de Monterrey, Campus Querétaro.